Alma Mater Studiorum-Università di Bologna
Scuola di Ingegneria

Lambda expression – Mini intro

Corso di Laurea in Ingegneria Informatica

Anno accademico 2025/2026

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA
IL PRESENTE MATERIALE È RISERVATO AL PERSONALE DELL'UNIVERSITÀ DI BOLOGNA E NON PUÒ ESSERE UTILIZZATO AI TERMINI DI LEGGE DA ALTRE PERSONE O PER FINI NON ISTITUZIONALI

LAMBDA EXPRESSION: AGENDA

ADESSO

- mini-introduzione, propedeutica alla grafica

DOPO LA GRAFICA

- approfondimento e trattazione più completa ed esaustiva

FUNZIONI IN MATEMATICA vs. IN C o JAVA

- In matematica, le funzioni si definiscono stabilendo *dominio e codominio*:

$$\mathbf{f: Z \rightarrow Z} \qquad \mathbf{g: R \rightarrow R} \qquad \mathbf{h: RxR \rightarrow R}$$

$$x \rightarrow 2x+1 \qquad x \rightarrow 2\sin(x/2) \qquad x,y \rightarrow \sqrt{x^2 + y^2}$$

- Tuttavia, nei linguaggi di programmazione solitamente si scrivono in modo diverso, come *blocco di codice* che *ritorna un risultato:*

```
public static int f(int x) { return 2*x+1; }
public static double g(double x) { return 2*Math.sin(x/2); }
public static double h(double x, double y) { return Math.sqrt(x*x + y*y); }
```

FUNZIONI IN MATEMATICA vs. IN C o JAVA

- In matematica, le funzioni si *applicano* a un valore:

$$\mathbf{f}(5) \quad \mathbf{g}(\pi/2) \quad \mathbf{h}(3.0, 4.0)$$

intendendosi che il valore specificato è sostituito alla variabile segnaposto:

$$\mathbf{f: Z \rightarrow Z} \qquad \mathbf{g: R \rightarrow R} \qquad \mathbf{h: RxR \rightarrow R}$$

$$5 \rightarrow 2 \bullet 5+1 \qquad \pi/2 \rightarrow 2 \bullet \sin(\pi/4) \qquad 3.0,4.0 \rightarrow \sqrt{3^2 + 4^2}$$

- Lo stesso avviene nei linguaggi di programmazione:

```
System.out.println( f(5) );          // 11
System.out.println( g(Math.PI/2) ); // 1.414
System.out.println( h(3,4) );        // 5.0
```

FUNZIONI NEI LINGUAGGI DI PROGRAMMAZIONE

- In effetti tutti i linguaggi di programmazione hanno costrutti per esprimere *funzioni,* ma nei linguaggi imperativi tradizionali una funzione è *solo un costrutto che incapsula codice*
  - ha un nome, un'intestazione (signature), un corpo
  - viene *chiamata* con l'operatore **()**

```java
public class Main {

    public static int f(int x) { return 2*x+1; }
    public static double g(double x) { return 2*Math.sin(x/2); }
    public static double h(double x, double y) { return Math.sqrt(x*x + y*y); }

    public static void main(String[] args) {
        System.out.println( f(5) );          // 11
        System.out.println( g(Math.PI/2) ); // 1.414
        System.out.println( h(3,4) );        // 5.0
    }
}
```

```
11
1.414213562373095
5.0
```

FUNZIONI NEI LINGUAGGI DI PROGRAMMAZIONE

- Ciò riflette però una visione «antica», in cui un programma è costituito da *dati* e *codice* intesi come *due mondi distinti*
  - i dati (semplici o strutturati) rappresentano valori o variabili da manipolare
  - il codice (immutabile) rappresenta il «comportamento»
  - in alcuni processori, non a caso, dati e codice erano proprio posti in *zone di memoria distinte* («segmenti»), *ben separate* fra loro
- Ma.. è davvero «sana», giusta, questa distinzione?
- Esiste però una intrigante possibilità alternativa: vedere le funzioni come un particolare «oggetto», di tipo «funzione»
  - come una Persona è caratterizzata da nome e cognome, una Funzione potrebbe avere come proprietà nome, elenco argomenti, tipo di ritorno e... codice!

FUNZIONI COME OGGETTI

- In questa ottica, una funzione diventa *un oggetto come gli altri,* ma con una proprietà in più: essere «*eseguibile*»!
  - a una Persona puoi chiedere nome, cognome, data di nascita, età
  - a una «Funzione» puoi chiedere nome, elenco argomenti, tipo di ritorno… ma anche di «mettersi in esecuzione»!
- Vedere la funzione come *un (particolare) oggetto* apre interessanti prospettive:
  - come ogni oggetto, potrà essere *assegnata* a riferimenti
  - come ogni oggetto, potrà essere *passata come argomento* ad altre funzioni, ottenendo così *funzioni del secondo, terzo, quarto ordine*
  - eccetera
- Ciò suggerisce di introdurre una *nuova sintassi leggera*

LAMBDA EXPRESSION

- Per definire oggetti-funzione si utilizza un *particolare tipo di espressione,* noto come *lambda expression,* del tutto analogo alla notazione matematica:

f: Z → Z	**g: R → R**	**h: RxR → R**
$x \rightarrow 2x+1$	$x \rightarrow 2\sin(x/2)$	$x,y \rightarrow \sqrt{x^2 + y^2}$

- Lambda expression corrispondenti:

Come esprimere che questi oggetti sono di tipo «funzione»…?

```
...   f = x -> 2*x+1;
...   g = x -> 2*Math.sin(x/2);
...   h = (x,y) -> Math.sqrt(x*x + y*y);
```

  – il nome deriva dal *lambda calcolo*, definito negli anni '30 da Church come formalismo alternativo alla Macchina di Turing per definire ciò che è computabile
  – esso utilizzava la lettera greca λ (lambda) per introdurre le funzioni

LAMBDA EXPRESSION

- Una *lambda expression* è una *funzione anonima,* espressa in modo simile alla notazione matematica:

  lista argomenti **->** *corpo della funzione*

  SINTASSI JAVA

  - se ci sono più argomenti, si elencano fra parentesi *tonde*
  - i *tipi* degli argomenti in generale *si omettono*, se deducibili dal contesto
  - se il corpo della funzione comprende più di un'espressione, va racchiuso fra parentesi *graffe*

Come esprimere che questi oggetti sono di tipo «funzione»…?

```
...   f = x -> 2*x+1;
...   g = x -> 2*Math.sin(x/2);
...   h = (x,y) -> Math.sqrt(x*x + y*y);
```

- Ma... *che tipo hanno* questi oggetti?

IL TIPO DELLA LAMBDA

- In matematica, *le funzioni non sono tutte dello stesso «tipo»*
  - NUMERO DEGLI ARGOMENTI: una funzione a un argomento non è intercambiabile con una a due o più argomenti
  - DOMINIO: una funzione sul dominio Z non è intercambiabile con una sul dominio R, o RxR, o altro
  - CODOMINIO: una funzione che restituisce un reale non è intercambiabile con una che restituisca una stringa
- Dunque, il «tipo» della funzione è definito dal suo *dominio* e dal suo *codominio*
- *Come esprimere ciò* in un linguaggio di programmazione?
  - in JAVA, l'idea è stata quella di *dare un nome* a ogni possibile tipo
  - serie di nomi standard + alcuni specifici per i tipi primitivi più usati (**`int`**, **`double`**, **`long`**)

IL TIPO DELLA LAMBDA IN JAVA

- Java adotta un approccio *NOMINALE*
  - ogni *tipo di funzione* ha un *nome*
  - serie di nomi standard + alcuni specifici per **`int`**, **`double`**, **`long`**
- In particolare:
  - quando dominio e codominio usano lo stesso tipo, *XXXOperator,*
    quando dominio e codominio usano tipi diversi, *Function*
  - operatori a un solo argomento si dicono *UnaryOperator,*
    operatori a due argomenti si dicono *BinaryOperator*
  - funzioni a un solo argomento si dicono *Function,*
    funzioni a due argomenti si dicono *BiFunction*
  - etc.

+ varianti specifiche per i tre tipi primitivi più usati
(es. **`IntBinaryOperator`**, **`IntFunction`**,…)

IL TIPO DELLA LAMBDA IN JAVA

- Java adotta un approccio *NOMINALE*
  - ogni *tipo di funzione* ha un *nome*
  - serie di nomi standard + alcuni specifici per **`int`**, **`double`**, **`long`**
- In particolare:
  - quando dominio e codominio usano lo stesso tipo, *XXXOperator*, quando dominio e codominio usano tipi diversi, *Function*
  - operatori a un solo argomento si dicono *UnaryOperator*, operatori a due argomenti si dicono *BinaryOperator*
  - funzioni a un solo argomento si dicono *Function*, funzioni a due argomenti si dicono *BiFunction*
  - etc

```
IntUnaryOperator     f = x -> 2*x+1;
DoubleUnaryOperator  g = x -> 2*Math.sin(x/2);
DoubleBinaryOperator h = (x,y) -> Math.sqrt(x*x + y*y);
```

DEFINIZIONI: CONFRONTO

- Funzioni definite in modo tradizionale:

```
int f(int x) { return 2*x+1; }
double g(double x) { return 2*Math.sin(x/2); }
double h(double x, double y) { return Math.sqrt(x*x + y*y); }
```

- Funzioni definite come lambda expression:

```
IntUnaryOperator     f = x -> 2*x+1;
DoubleUnaryOperator  g = x -> 2*Math.sin(x/2);
DoubleBinaryOperator h = (x,y) -> Math.sqrt(x*x + y*y);
```

- …e per quanto riguarda la chiamata?

CHIAMATA DI FUNZIONI

- In teoria, la chiamata *non* dovrebbe cambiare
  - la notazione matematica usa anch'essa la notazione *f*(*arg*) per applicare la funzione *f* a un dato argomento *arg*

In effetti, in quasi tutti i linguaggi (C#, Scala, Kotlin) è così

- Purtroppo, *non è così* in Java
  - MOTIVO: le lambda furono introdotte ponendo grande cura a non interferire con l'architettura pre-esistente
  - CONSEGUENZA: il tipo della lambda è in realtà una normale interfaccia, che racchiude un normalissimo metodo
    → purtroppo, *tale metodo risulta visibile nella chiamata* ☹☹
  - ulteriore inestetismo: il nome del metodo racchiuso non è fisso (in molti linguaggi è ***`apply`***) ma *varia da un'interfaccia all'altra* ☹☹

CHIAMATA DI FUNZIONI LAMBDA IN JAVA

- Schema (parziale) di naming
  - i vari *XXXOperator* usano un metodo di nome *applyAsXXX*
  - le varie *Function* usano un metodo di nome *apply*
- In particolare:
  - *IntUnaryOperator, IntBinaryOperator* usano *applyAsInt*
  - *LongUnaryOperator, LongBinaryOperator* usano *applyAsLong*
  - *DoubleUnaryOperator, DoubleBinaryOperator* usano *applyAsDouble*

```
IntUnaryOperator     f = x -> 2*x+1;
DoubleUnaryOperator  g = x -> 2*Math.sin(x/2);
DoubleBinaryOperator h = (x,y) -> Math.sqrt(x*x + y*y);

System.out.println( f.applyAsInt(5) );              // 11
System.out.println( g.applyAsDouble(Math.PI/2) );   // 1.414
System.out.println( h.applyAsDouble(3,4) );         // 5.0
```

CHIAMATE: CONFRONTO

- Chiamate di funzioni definite in modo tradizionale:

```
System.out.println( f(5) );          // 11
System.out.println( g(Math.PI/2) ); // 1.414
System.out.println( h(3,4) );        // 5.0
```

- Chiamate di funzioni definite come lambda expression:

```
System.out.println( f.applyAsInt(5) );              // 11
System.out.println( g.applyAsDouble(Math.PI/2) );  // 1.414
System.out.println( h.applyAsDouble(3,4) );        // 5.0
```

  – NB: questo inestetismo non è presente in altri linguaggi come C#, Scala, Kotlin

LAMBDA EXPRESSION: UTILITÀ

- Le lambda non sono solo più «belline» e «vicine alla matematica» delle funzioni tradizionali
- Il vero valore aggiunto è che *permettono di assegnare un oggetto-funzione a una variabile* e quindi anche di *passarlo come argomento* a un'altra funzione (o restituirlo)
- Ciò consente di:
  - rendere parametrico il comportamento di certe funzioni (librerie), *iniettando comportamento* per personalizzarle
    → possibilità di ripensare e riprogettare le librerie di largo uso
  - definire funzioni di ordine superiore

ESEMPIO: CALCOLATRICE

- Una funzione «calcolatrice» può essere definita ad esempio ricevendo *la specifica operazione* da svolgere e *lo specifico valore* su cui farla:

```
public static double calc(DoubleUnaryOperator op, double arg){
    return op.applyAsDouble(arg);
}
```

- Possibile uso:

```
DoubleUnaryOperator f1 = x -> 2*Math.sin(x/2);
DoubleUnaryOperator f2 = y -> Math.sqrt(y);
DoubleUnaryOperator f3 = x -> (x+3)/(x-2);

System.out.println( calc(f1, Math.PI) );        // 2
System.out.println( calc(f2, 1.0/9.0) );        // 0.333
System.out.println( calc(f3, 3.0)     );        // 6
```

  – **`calc`** calcola *la specifica funzione ricevuta (f1, f2 f3)* in corrispondenza del valore indicato come secondo argomento.

ESEMPIO: CALCOLATRICE

- Il bello è che possiamo chiamare calc *anche con funzioni lambda che definiamo sul momento,* non necessariamente solo su funzioni definite in precedenza:

```
System.out.println( calc( z->z+1,  4.5) );                  // 5.5
System.out.println( calc( x->x*10, 4.4) );                  // 44
System.out.println( calc( r -> Math.sin(r), Math.PI/4) );   // 0.707
```

CALCOLATRICE: VARIANTE (poor)

- Anche la funzione «calcolatrice» potrebbe essere definita come lambda, ma a causa delle limitazioni di Java il risultato sarebbe poco leggibile:
  - **`calc`** è una funzione a due argomenti (*BiFunction*), il cui primo argomento è a sua volta un operatore (*DoubleUnaryOperator*), il secondo argomento è un *Double* (wrapper per *double*) e il risultato è pure un *Double* (altro wrapper per *double*)

```
public static BiFunction<DoubleUnaryOperator,Double,Double> calc
                = (op,arg) -> op.applyAsDouble(arg);
```

  - oltre tutto, nell'uso diventerebbe visibile il metodo interno apply:

```
System.out.println( calc.apply(f1, Math.PI) );        // 2
System.out.println( calc.apply(f2, 1.0/9.0) );        // 0.333
System.out.println( calc.apply(f3, 3.0)     );        // 6
```

RIFERIMENTI A METODI (METHOD REFERENCE)

- Se la funzione da chiamare esiste già altrove e ci limitiamo a chiamarla «così com'è», può far comodo una scorciatoia, il *riferimento a metodo (method reference),* che evita di scrivere l'argomento e perfino la chiamata:

`NomeClasse::nomeMetodo`

- Ad esempio, per chiamare *Math.sin*, anziché scrivere:

```
System.out.println( calc( r -> Math.sin(r), Math.PI/4) );   // 0.707
```

- si può più compattamente scrivere:

```
System.out.println( calc(Math::sin, Math.PI/4) ); // 0.707
```

  – questo ci farà molto comodo nella grafica e nei comparatori

RIFERIMENTI A METODI (METHOD REFERENCE)

- Ad esempio, se nella classe *Main* ci fosse un'altra funzione (statica) *g:*

```
private static double g(double w) { return w*w-4; }
```

- potremmo passarla a *calc* sia in forma estesa, sia come *method reference:*

```
System.out.println( calc( x -> g(x), 3.0) );    // 5.0, forma estesa
System.out.println( calc(Main::g,    3.0) );    // 5.0, method reference
```

RIFERIMENTI A METODI (METHOD REFERENCE)

- Un altro caso tipico nella grafica è l'aggancio di un metodo a un pulsante, che si fa passando il metodo a *setOnAction* (l'analogo di calc dell'esempio precedente):

```
public class EsJavaFX08ter extends Application {
  public void start(Stage stage){
    …
    Button b = new Button("Tizio/Caio");
    b.setOnAction( ev -> myHandle(ev) );
    …
  }
  void myHandle(ActionEvent event) {
    l.setText( l.getText().equals("Tizio") ? "Caio" : "Tizio");
  }
}
```

RIFERIMENTI A METODI (METHOD REFERENCE)

- In questo caso però la funzione da chiamare *non è più una funzione statica*, ma un metodo di istanza
- Il prefisso da usare non è allora più il nome della classe, bensì quello dell'*oggetto contenitore* – qui, *this*:

```
public class EsJavaFX08ter extends Application {
   public void start(Stage stage){
      …
      Button b = new Button("Tizio/Caio");
      b.setOnAction(this::myHandle);
      …
   }
   ...
}
```